

Synchronizing Application State Using Virtual DOM Trees

Jari-Pekka Voutilainen¹(✉), Tommi Mikkonen², and Kari Systä²

¹ Gofore Ltd., Hämeenkatu 16, 33200 Tampere, Finland
`jari.voutilainen@iki.fi`

² Tampere University of Technology, Korkeakoulunkatu 1, 33720 Tampere, Finland
`{tommi.mikkonen,kari.systa}@tut.fi`

Abstract. We will all soon have numerous computing devices we use every day interchangeably. Liquid software, a concept where software is allowed to flow from one computer to another, is a programming framework that aims at simplifying the development and use of such multi-device software. The existing research has discovered three major architecture challenges for liquid software: (1) adaptation of the user interface to different devices, (2) availability of the relevant data in all devices, and (3) transfer of the application state. This paper addresses the last challenge and differs from the earlier work by concentrating in application state that is in the DOM tree, a key element in today's Web applications.

Keywords: Web programming · Multi-device ownership · Experience roaming · Liquid software

1 Introduction

Managing computers and software that runs in them is complicated when using multiple devices interchangeably. While multi-device programming has been proposed [1], we assume using PCs and mobile devices as the primary devices, with the need to maintain each of them separately and independently of each other [2]. This includes both data and applications that are run in the devices. While cloud computing helps with data, reinstalling applications, setting up accounts, and other complications will however plague our computing experience for years to come.

Liquid software, a concept where software is allowed to flow from one computer to another [3–5], is a programming framework that aims at simplifying the development of multi-device software. Instead of treating applications as device-specific nuisance that must be installed to all of them, the framework enables switching devices while still running the same application. Furthermore, we assume that it is thus achieved using only web technologies [6].

In this paper, we extend the technology basis [7] with a new DOM tree-based synchronization between different browsers. The paper is structured as

follows. In Sect. 2, we discuss the earlier work and background of this paper. In Sect. 3 we define the goals of the framework. In Sect. 4 we introduce the new implementation, building on DOM trees. In Sect. 5 we look at our example application. In Sect. 6, we provide an extended discussion regarding the lessons learned regarding the design, and draw some final conclusions.

2 Background and Motivation

Experiences gained with numerous experiments, including for example Cloudberry [8], HTML5 Mobile Agents [9], and Cloud Browser [10], led us to consider a different type of computing paradigm, where applications are not artificially tied to a particular computer. Instead, they could flow from one computer to another, in a fashion that is casual and free from hassle from end-user perspective – or, as put in our earlier papers, in a liquid fashion [5]. Our earlier research has motivated us to build a application framework that would make development of liquid applications easier. The same experiments showed us that rich, powerful and especially portable applications can be implemented by using standard Web technologies like HTML, CSS and JavaScript.

The software framework for Liquid applications includes three main subsystems [6]:

- Adaptation of the user interface to different devices and contexts. While the functions of an application may remain the same, the devices used to run the application may differ in terms of display and input technologies. In addition, the users’ ability and interest to pay constant attention differ according to the context.
- Synchronization of the users’ persistent data and content. The applications need to have an access to files like pictures, documents and PIM (Personal Information Management) data. This content is commonly stored in cloud-based services instead of the local storage of the devices. Unfortunately, these cloud-based storage systems are generally limited to single applications, Internet services, or to ecosystems built on certain operating systems. Some research like data API of Cloudberry mentioned above and EDB [11] address these problems by providing programming APIs and efficient synchronization mechanisms. Similar functionality could also be achieved by storing data in BaaS (Backend-as-a-Service) systems like Firebase¹ but those systems are not optimized for files.
- Synchronization of the state of the applications that move during execution or for applications that run on multiple devices at the same time. This state information includes values of relevant variables and control state. In case of Web applications the state could include JavaScript variables and content in the DOM tree. Since the execution model of Web applications is based on reasonable short events handlers, and especially if we assume that Web Workers are not used and applications should not move during incomplete I/O operation, no control state has to be included.

¹ <https://www.firebase.com/>.

In this paper we concentrate on the last item, i.e., synchronization of the application state and place the focus on technology that is needed to implement state synchronization. Other topics have been addressed in our earlier work [2,5,6,12]. While designing a architecture for state synchronization, the level of synchronization needs to be decided. These levels would range from lightweight variable transfer to complete application transfer. The most complete mechanism of migrating application state from one computer to another would be to transfer the complete memory space of the application. However, this is not technically feasible for a number of reasons:

- The amount of data to be moved is too big for almost any non-trivial application.
- Internal data structures, stored in the memory, cannot be moved as such, since they depend on underlying platform, as well as browser and virtual machine implementation.
- Present browser implementations do not support direct, serialized memory dumps.
- The application specific needs are not taken into account. In reality the set of synchronized data depends on the application.
- The content of the memory depends also on the browser implementation, and complex conversion mechanisms would be needed.

Over time, we have been working on several approaches for synchronization of the application state. All these approaches have their strengths and weaknesses. The approaches and their characteristics will be addressed next.

In Lively3D [13] we designed a 3D environment within browser window, where applications were rendered to 3D objects. The framework included utilities for serialization and deserialization of Javascript objects that contained parts of application state. The serialized object was migrated to another browser through Dropbox or MongoDB database. In this framework the developer had to define the structure and content of JavaScript state objects as well as the concrete serialization and deserialization functions.

In Cloudberry [8], Cloudbrowser [10], and in some student projects, we have used BaaS based (Backend-as-a-Service) synchronization – either proprietary solutions built for the exact purpose, like in Cloudberry, or publicly available BaaS systems. Although these experiments have shown that the applications can be made movable with these solutions, the approach assumes that the applications uses the BaaS system explicitly store the critical state information.

In HTML5 Mobile agents [9] the applications mark the JavaScript variables to be synchronized. The values of the marked variables are then automatically serialized and migrated as part of the moving agent. This approach works well for cases where applications are not run simultaneously and the DOM tree is not used for storing relevant state information. Unfortunately, many programming paradigms and toolkits assume that part of the state is stored in DOM. These limitations lead us to think about a solution that can provide more automatic synchronization of data and also include data in the DOM tree.

Another design decision is the communication topology. In many of our experiments we have used a centralized topology. In that approach all state updates are stored in a centralized server from which other instances receive the state updates. In cases where applications are run simultaneously on multiple devices some peer-to-peer mechanisms could be used for synchronization, too. In our initial experiments we have used WebRTC [14] to implement a peer-to-peer synchronization routine. This approach is not applicable to cases where applications are not running on multiple devices simultaneously. Furthermore, such approach faces some implementation problems due to network infrastructures; for instance, NAT (Network Address Translation) often make use of the peer-to-peer approach difficult.

As listed above there are several approaches to implement roaming of the applications from device to another. The options differ from each other in several ways:

- support for simultaneous, sequential or both scenarios;
- need to have a centralized server;
- affect to programming model, where the state can be, selection of UI libraries, and other constraints.

3 Goals and the Core Idea

This work is a part of larger research aiming at a complete application framework for liquid applications where the goal is to minimize the additional work by the developer. Thus, the requirements for the framework are:

- Development of applications for the framework should be simple. The developer should not need to design the application to be Liquid and most of the existing applications should be able to use the framework in some extent.
- The framework should be based on existing browser implementations. The user should not need to install a custom browser or fiddle with the settings. Any modern browser should be usable as long as it supports the Web Standards used in the framework.
- The applications using the framework should be device agnostic. Part of the Liquid Software Manifesto is multi-device ownership, Liquid applications should be usable in desktops, mobile phones and tablets. Therefore the applications using the framework must be usable also in mobile devices and the migration of applications state should not differentiate between these.

The application state of any modern client-side web application is stored in both JavaScript namespace and the DOM tree of the browser window. This may be result of explicit choice by the developers or consequence of the libraries used in the implementation. We see that a system that can serialize content from both JavaScript variables and DOM tree provides the most flexible solution for liquid web applications since it supports wide range of different frameworks and programming styles. In the work reported in this paper we complement our

earlier work on serializing state in the JavaScript variables with techniques to migrate parts of the DOM tree from one browser window or device to another and restoring it to the same state.

Although the browsers provide JavaScript APIs to read the content of the DOM tree, copying of the whole tree after each state change is not feasible. The DOM tree of the application is often large and most of it remains unchanged during execution of application. In most applications some content of the DOM tree also reside in JavaScript variables, which could be used to restore the DOM tree at least in some extent. Without designing the application to store all the DOM content in JavaScript variables, relying only to the JavaScript variables is not feasible to transfer DOM tree states.

This paper proposes a new mechanism to serialize data information from the DOM tree by using a concept called *Virtual DOM trees*. Virtual DOM is a novel technology popularized by React.js². The basic idea of Virtual DOM is that rather than touching the DOM tree directly, application builds an abstract version of the tree. This abstraction makes DOM manipulation significantly faster when is coupled with efficient comparison algorithms and operation on selective sub-trees. Other implementations of Virtual DOM include virtual-dom³, Mithril⁴ and Bobril⁵.

Virtual DOM enables manipulation of the DOM trees in JavaScript namespace. We can create virtual DOM trees in JavaScript variables, modify them, compare two different virtual trees and apply these comparisons back to the actual DOM tree. Similar technology under standardization is Shadow DOM [15], which encapsulates multiple DOM trees within one document. However, this technology does not provide means to compare the current DOM tree to another tree and therefore it cannot be used in our approach as such.

4 Design and Implementation

We present our proof-of-concept implementation of the Liquid Application Framework, later named Liquid.js. Liquid.js provides APIs for application developer to use, to enable DOM tree migrations in easy to use manner. It should be noted that our ultimate goal is a complete Liquid.js that supports several aspects of making application liquid, but the proof-of-concept discussed in this paper concentrates on the reported mechanism for state synchronization.

The cornerstone of our approach is the initial state of the DOM trees. When an application is loaded and the browser throws a *load* event, the initial state is formed as a Virtual DOM tree and stored within the framework for further use. Later, during migration, we only transfer differences between the original Virtual DOM tree and the current DOM to minimize the amount of transferred data. When a migration of a state is initiated either by a user action or by some

² <https://facebook.github.io/react/docs/glossary.html>.

³ <https://github.com/Matt-Esch/virtual-dom>.

⁴ <http://mithril.js.org/>.

⁵ <https://github.com/Bobris/Bobris>.

external event, the current state of the DOM tree is compared against the stored initial state using comparison algorithm provided by the *virtual-dom* -library. This comparison produces a set of patch operations that encapsulates the needed modifications from the initial DOM tree to the current tree. To minimize the data needed for migration, the prototype objects (named *VirtualNode*, *VirtualPatch* and *VirtualText*) are removed before sending and re-created after migration in the new browser.

After the prototypes have been recreated, the received patch operations are applied to the initial state virtual DOM tree. This tree is then applied to the actual DOM tree. This results in exactly the same state as in the origin browser. This process is also presented in Fig. 1.

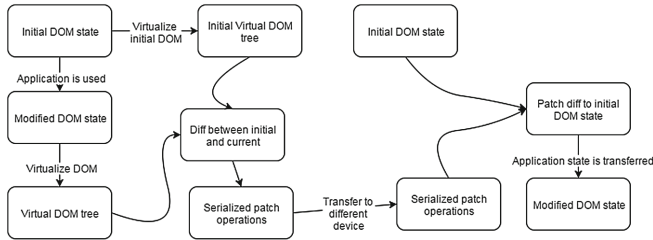


Fig. 1. Serialization of DOM tree to different browsers.

Since the patch operations assume the commonly known initial state of the DOM tree, the destination browser needs to be moved back to that initial state before applying the patches. After the destination browser is in known initial state, the deserialized migration object with the recreated prototype objects is applied to the DOM tree which results to the synchronized DOM state between the browsers.

There are several options for the architecture and implementation of the network. In our proof-of-concept the synchronization is based on minimal store-and-forward server and WebSockets for inter-browser communication. The centralized server is the bare minimum needed for the migration to work. It provides functions to query the browsers that have initialized Liquid.js applications and to transfer migration objects between browsers. The migration objects can be sent to single browser or all of them.

Our demo application, presented in detail in Sect. 5, is loaded into multiple browser windows in different devices. In each browser the initial state is stored during the initialization of Liquid.js. In our demo, the migrations are triggered by user action, but since Liquid.js provides JavaScript function for triggering the migration, almost any browser event could be used for it.

Due to the discarding of the current state during migration in receiving browser, Liquid.js essentially implements the sequential paradigm of Liquid Software. But because the application is not removed from the source browser, simultaneous paradigm could be implemented with migrating every time the state

changes in one of the browsers. For fully implementing simultaneous screening, merging of the states should be implemented instead of reverting to the known initial state. Examples of both scenarios are provided in Figs. 2 and 3, where state is migrated between three different browsers. In Fig. 2, the migrations are user triggered, and there, the user chooses the destination browser.

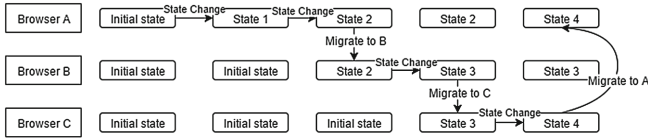


Fig. 2. Sequential migration where migrations are user triggered.

In Fig. 3, the migrations are triggered by the changes and the migrations are transferred to every browser.

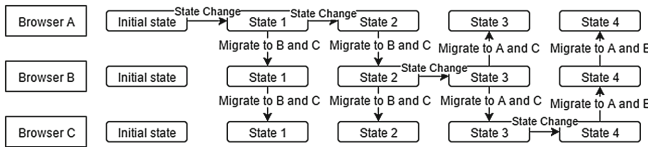


Fig. 3. Simultaneous migration where migrations are triggered by state changes.

The migration object formed from the Virtual DOM tree contains information only found from the real DOM tree, for example class names, identifiers and identities, values, and other attributes. Anything related to the DOM tree, but actually residing in the JavaScript namespace will not get serialized with the DOM tree. Most common case is event listeners that are bound to the DOM elements. After migration the DOM tree would be the same, but the functionality bound to the DOM elements would have disappeared. An example of the DOM tree has been provided in Fig. 4. Everything not present in the example DOM elements, are not migrated with virtual DOM.

Since only the data residing in the DOM tree is migrated through virtual DOM trees, Liquid.js has functionality to register event listeners that should be serialized with the DOM tree and rebound during creation of DOM elements. This was implemented so that application does not need to pollute the global scope with function names that could be visible in a much smaller scope. Example of application that uses Liquid.js to enable migration of DOM trees is provided in Listing 1. This listing presents only the lines of code needed to enable successful migration and it uses the default implementations to simplify the example.

```

▶ <div class="row">...</div>
▶ <div class="row">...</div>
  <hr>
  ▼ <ul class="list-group todo-list">
    ▼ <li class="list-group-item">
      ▶ <input type="checkbox" class="toggle" data-handler="toggleClick">...</input>
      <label class="done">item 1</label>
      <button class="destroy" data-handler="destroyClick"></button>
    </li>
    ▶ <li class="list-group-item">...</li>
    ▶ <li class="list-group-item">...</li>
  </ul>
  ::after
</div>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.3/jquery.min.js"></script>
<script src="bootstrap/dist/js/bootstrap.min.js"></script>

```

Fig. 4. A part of the DOM tree as seen in chrome dev tools.

```

var Liquid = require('liquid.js');
var liquid = new Liquid();

var checkbox = document.createElement('input');
checkbox.addEventListener('click', someFunction);
checkbox.setAttribute('data-handler', 'toggleClick');

liquid.registerHandler('toggleClick', {'click': someFunction});

```

Listing 1: Example application using Liquid.js

Liquid.js consists of multiple modules designed to handle different aspects of the framework. These modules interact with each other and third party developed open source modules, which each have their own dependencies. In Fig. 5 a graph of the modules is presented with explanations if the module is implemented with the Liquid.js framework or if it is developed by third party. Each box in the figure is a single module and the arrows describe dependencies between them. Third party modules are available in npm⁶ and Github⁷.

The description of modules is given in following:

- *Liquid.js* is the the main module, which encapsulates the functionality of other modules. This also provides interfaces for the application developer.
- *Liquid.js / UI* provides optional User Interface elements, which can be used to control migrations.
- *Liquid.js / Variables* contain functions for variable and function registrations. These are required to enable rebounding of functions after migration.
- *Liquid.js / VDOM* contains Virtual DOM tree related functions.
- *vdom-serialize* provides functionality for serialization and deserialization of Virtual DOM trees.
- *vdom-virtualize* enables Virtual DOM tree creation from DOM tree.
- *virtual-dom* handles Virtual DOM related algorithms: comparison and merging Virtual DOM to DOM tree.

⁶ <https://www.npmjs.com/>.

⁷ <https://github.com/>.

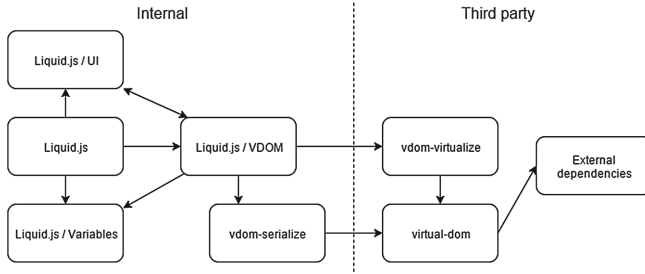


Fig. 5. Architecture of Liquid.js. Each box is a module available in npm or Github.

Even though Liquid.js is built using WebSockets, practically any communication mechanisms could be used as the communication channel for the state migration. The constructed data structure is a standard JSON object, which is serialized by Liquid.js, and it could be transferred in a number of ways, including server-side databases, BaaS solutions, and even WebRTC. The framework is designed to support other transfer methods and the application developer can choose which one to use. By choosing different transfer methods, application developer can implement more or less real-time synchronizations for different needs of applications.

The application needs to conform to the limitations of DOM trees and APIs that Liquid.js provides for enabling state migration. If the following requirements are fulfilled, the migration will be successful without any complications:

- The DOM tree is built so that its state is applicable for migration as it is at any given time.
- Event handler functions and variables that are not in global scope, are required to be registered to Liquid.js.
- The receiving browser's Liquid.js reverts application to initial state before applying the migrating application state to avoid merging complications which are out of scope of Liquid.js.

Responsive user interfaces are out of scope of Liquid.js, but they can be used in applications if the requirements listed above are met. For example Bootstrap⁸ and Foundation⁹ both implement responsive UIs using CSS classes, which are stored within the DOM tree, so these would work without a problem.

5 Example Application

Our example application is a basic todo list application. It has functionality for adding items, marking them done and removing them completely. All the application state is stored within the DOM tree for the sake of the demo, the JavaScript namespace does not contain anything that is not present right

⁸ <http://getbootstrap.com/>.

⁹ <http://foundation.zurb.com/>.

after application initialization. Whole application is built using Browserify¹⁰ which enables using require function in frontend code. A screenshot and application structure is presented in Fig. 6.

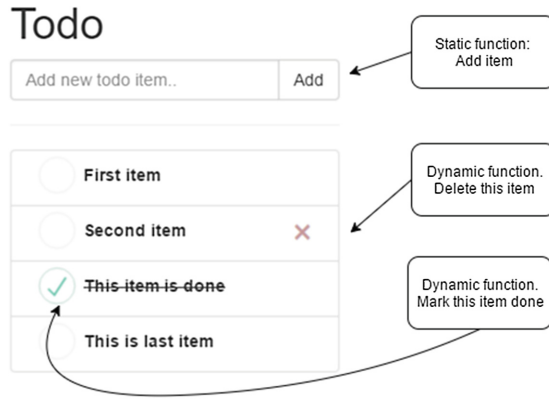


Fig. 6. Demo application and its functions without Liquid.js.

The liquid framework need to be initializes before use as presented in Listing 2. By default this initializes WebSockets for communication and adds user interface elements for management of the migrations. During initialization the initial state is also stored for future use, so Liquid.js should be initialized in *load*-event which is thrown by the browser when the application has finished loading. This enables virtual DOM migrations between browsers.

```
var Liquid = require('liquid.js');
var liquid = new Liquid();
```

Listing 2: Initialization of Liquid.js

As stated above while migrating only the virtual DOM, the bindings to dynamic JavaScript event handlers are lost since they are not residing in the DOM. These need to be registered to Liquid.js, so that they are serialized with the virtual DOM. In registration Liquid.js needs to know in what event it should bind the function after migration. Registered event handlers are given a name in the DOM, this is saved to the *data-handler* attribute of the DOM element. Example of these is provided in Listing 3.

```
// Add data-handler="removeItem" to removeButton
removeButton.setAttribute('data-handler', 'removeItem');

// Register removeTodo event handler to bind
//click-event to DOM element with data-handler removeItem
liquid.registerHandler('removeItem', {'click': removeTodo});
```

Listing 3: Registration of event handler for serialization.

¹⁰ <http://browserify.org/>.

While migrating Liquid.js removes its own user interface elements, since they are not part of the application and we do not want to migrate those. After removing, Liquid.js serializes the differences in DOM compared to the initial state and the registered dynamic functions. When this has been migrated to another browser, the differences are applied to the DOM and event handlers are rebound to the DOM elements. Finally the Liquid.js user interface elements are recreated. Figure 7 presents screenshot of the application with Liquid.js user interface and their removal during migration.

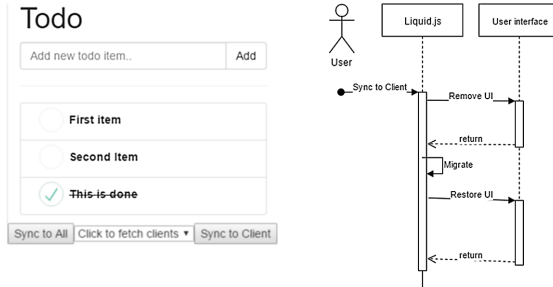


Fig. 7. User interface elements of Liquid.js and their removal during migration.

6 Conclusions

The emerging paradigm of programmable world, where all our objects act as one [16], means that all the devices can no longer be treated as individual computers – all their data, content, and programs simply cannot be installed separately in each device by the end user. Instead, we need a new type of programming paradigm we call liquid software, where applications from one device can migrate to another one, and continue their execution in the new context.

In this paper, we propose a framework for liquid web applications. The framework is based on synchronizing DOM trees to migrate the state of a web application, and it contrasts our earlier experiments [2, 5, 6, 12] by proposing a direction that is closer in spirit to web designs than programming approaches. While the presented implementation is based on WebSockets, this is not an essential design restriction, as (almost) any other protocol could be used as the transport mechanism.

Liquid.js implements proof-of-concept prototype of one part of the liquid software vision. It demonstrates that storing vital information of application to the DOM tree and migrating the tree to another browser is possible and without the need of rebuilding the application by the developer. In numerous practical cases, applications would not solely rely on either DOM trees nor JavaScript namespace to store the state of the application. In practice applications would store some of the state to the DOM tree and some to JavaScript namespace.

During the implementation of Liquid.js, it was found out that in the current web programming model, the DOM tree, and JavaScript namespace are deeply mingled. Since virtual DOM only interacts with the DOM tree, all the information stored in JavaScript namespace that were not there during the initialization of application, were lost during migration in our early experiments. We needed a functionality to specify important variables and functions to be serialized with the DOM tree, so that after migration the application would be practically the same with state and functionality.

Our design implements migration of applications with minimal overhead to application code. With only few functions to call when registering relevant variables and functions related to the DOM tree, it is simple to enable migration while developing applications. Even though there probably still are some issues to be solved in our proof of concept, Liquid.js presents DOM trees as viable option to store application state in Liquid Software. It has been common practice as long as there have been web applications and it should not be a problem in Liquid Software either. All code for our proof-of-concept is available at <https://github.com/Zharktas/liquid.js> and for the demo application at <https://github.com/Zharktas/Liquid-Todo>.

In further research we would develop more complete solution where we would implement results from our previous findings to a single application framework. This would include DOM tree migrations, mechanisms for JavaScript namespace transfer and merging of application states in simultaneous screening instead of replacement.

Related studies have been made: in [17] Bellucci et al. researched serialization of JavaScript namespace and implemented serialization functions for multiple standard JavaScript objects like timers and dates. In [18] Ghiani et al. proposed a proxy server implementation which was used to transfer browser session from user to user. Finally in [19] Klokmoose et al. implemented framework called Webstrates which executed web applications in iframe elements and the parent frame handled state synchronization between browsers.

Acknowledgments. This work has been partially supported by Academy of Finland (projects 283276 and 295913).

References

1. Mäkitalo, N., Pääkkö, J., Raatikainen, M., Myllärniemi, V., Aaltonen, T., Leppänen, T., Männistö, T., Mikkonen, T.: Social devices: collaborative co-located interactions in a mobile cloud. In: Proceedings of the 11th International Conference on Mobile and Ubiquitous Multimedia, p. 10. ACM (2012)
2. Taivalsaari, A., Mikkonen, T.: From apps to liquid multi-device software. *Proc. Comput. Sci.* **56**, 34–40 (2015)
3. Hartman, J., Manber, U., Peterson, L., Proebsting, T.: Liquid software: a new paradigm for networked systems. Technical report, Technical report 96 (1996)
4. Hartman, J.J., Bigot, P., Bridges, P., Montz, B., Piltz, R., Spatscheck, O., Proebsting, T., Peterson, L.L., Bavier, A., et al.: Joust: a platform for liquid software. *Computer* **32**(4), 50–56 (1999)

5. Taivalsaari, A., Mikkonen, T., Systä, K.: Liquid software manifesto: the era of multiple device ownership and its implications for software architecture. In: 2014 IEEE 38th Annual Computer Software and Applications Conference (COMPSAC), pp. 338–343. IEEE (2014)
6. Mikkonen, T., Systä, K., Pautasso, C.: Towards liquid web applications. In: Cimini, P., Frasin, F., Houben, G.-J., Schwabe, D. (eds.) ICWE 2015. LNCS, vol. 9114, pp. 134–143. Springer, Heidelberg (2015)
7. Gallidabino, A., Pautasso, C., Ilvonen, V., Mikkonen, T., Systä, K., Voutilainen, J.P., Taivalsaari, A.: On the architecture of liquid software: technology alternatives and design space
8. Taivalsaari, A., Systä, K.: Cloudberry: an HTML5 cloud phone platform for mobile devices. *IEEE Softw.* **29**(4), 40–45 (2012)
9. Systä, K., Mikkonen, T., Järvenpää, L.: HTML5 agents: mobile agents for the web. In: Krempels, K.-H., Stocker, A. (eds.) WEBIST 2013. LNBIP, vol. 189, pp. 53–67. Springer, Heidelberg (2014)
10. Taivalsaari, A., Mikkonen, T., Systä, K.: Cloud browser: enhancing the web browser with cloud sessions and downloadable user interface. In: Park, J.J.J.H., Arabnia, H.R., Kim, C., Shi, W., Gil, J.-M. (eds.) GPC 2013. LNCS, vol. 7861, pp. 224–233. Springer, Heidelberg (2013)
11. Koskimies, O., Wikman, J., Mikola, T., Taivalsaari, A.: EDB: a multi-master database for liquid multi-device software. In: Proceedings of the Second ACM International Conference on Mobile Software Engineering and Systems. MOBILESoft 2015, pp. 125–128. IEEE Press, Piscataway (2015)
12. Voutilainen, J.P., Salonen, J., Mikkonen, T.: On the design of a responsive user interface for a multi-device web service. In: Proceedings of the Second ACM International Conference on Mobile Software Engineering and Systems, pp. 60–63. IEEE Press (2015)
13. Voutilainen, J.P., Mattila, A.L., Mikkonen, T.: Lively3D: building a 3D desktop environment as a single page application. *Acta Cybernetica* **21**, 291–306 (2014)
14. W3C WebRTC Working Group: WebRTC 1.0: real-time communication between browsers (2016)
15. W3C Web Platform Working Group: Shadow DOM (2015)
16. Wasik, B.: In the programmable world, all our objects will act as one. *Wired* (last modified 14 May 2013), p. 462 (2013)
17. Bellucci, F., Ghiani, G., Paternò, F., Santoro, C.: Engineering javascript state persistence of web applications migrating across multiple devices. In: Proceedings of the 3rd ACM SIGCHI Symposium on Engineering Interactive Computing Systems, pp. 105–110. ACM (2011)
18. Ghiani, G., Paternò, F., Santoro, C.: Push and pull of web user interfaces in multi-device environments. In: Proceedings of the International Working Conference on Advanced Visual Interfaces, pp. 10–17. ACM (2012)
19. Klokmoose, C.N., Eagan, J.R., Baader, S., Mackay, W., Beaudouin-Lafon, M.: Webstrates: shareable dynamic media. In: Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology. UIST 2015, pp. 280–290. ACM, New York (2015)